

Lecture 6 Greedy Algorithm

Content

- Applications of Greedy algorithms
 - minimum spanning trees
 - Huffman encoding
 - set cover
- Optimality proof
 - optimal substructure + greedy choice
 - matroid

Characteristics of greedy algorithm

- efficient
- easy to implement
- not guarantee an optimal solution
- Often used as an approximated algorithm

Minimum Spanning Trees

Input: An undirected graph $G = (V, E)$, edge weights W_e

Output: A tree $T = (V, E')$, $E' \subseteq E$ that minimizes $\text{weight}(T) = \sum_{e \in E'} W_e$.

(MST is not unique)

A Greedy Approach

Kruskal's MST algorithm:

- Start with the empty graph
- Repeatedly add the next lightest edge that doesn't produce a cycle.

Input: a connected undirected graph $G=(V,E)$ with edge weights

Output: A minimum spanning tree defined by the edges X

```
for all u in V:
```

```
    makeset(u) /* create a set only containing u, i.e., each vertex is a
connected componenet */
```

```
X = {}
```

```
Sort the edges E by weight
```

```

for all edges {u,v} in E, in increasing order of weight:
    if find(u) != find(v) /* check whether u and v are in a same set(whether
is connected) */
        add edge {u,v} to X
        union(u,v) /* merge the sets containing x and y */

```

$|V|$ makeset, $2|E|$ find, $|V| - 1$ union.

The total time: $O((|V| + |E|) \log |V|)$ if implemented with disjoint sets. (the main factor is sort.

$O(E \log E) \implies O(E \log V) \implies O((|V| + |E|) \log |V|)$.

Disjoint Set

Disjoint sets contain the nodes of a particular component. We use a directed tree to store disjoint trees. Node of tree are elements of the set. Each node has a parent pointer and eventually leads up to the root of the tree.

Each node has a parent pointer π and a rank (the height of the subtree rooted at that node).

- makeset(x) : $O(1)$.
- find(x) : Follow the parent pointer to the root of the tree. $O(\log n)$.
- Union(x, y) : make the root of shorter tree point to the root of the taller tree. The overall height increase only if equally tall. $O(\log n)$.

procedure makeset(x)

$\pi(x) = x$

$\text{rank}(x) = 0$

function find(x)

while $x \neq \pi(x)$: $x = \pi(x)$

return x

procedure union(x, y)

$r_x = \text{find}(x)$

$r_y = \text{find}(y)$

if $r_x = r_y$: return

if $\text{rank}(r_x) > \text{rank}(r_y)$:

$\pi(r_y) = r_x$

else:

$\pi(r_x) = r_y$

if $\text{rank}(r_x) = \text{rank}(r_y)$: $\text{rank}(r_y) = \text{rank}(r_y) + 1$

Properties:

- For any $x \neq \pi(x)$, $\text{rank}(x) < \text{rank}(\pi(x))$
- Any root node of rank k has at least 2^k nodes in its tree.
- If there are n elements overall, there can be at most $n/2^k$ nodes of rank k .

The Cut Property

Cut: any partition of the vertices into two groups, S and $V - S$.

Cut property: Suppose edges X are part of a MST of $G = (V, E)$. Pick any subset of nodes S for which X does not cross between S and $V - S$ and let e be the lightest edge across this partition. Then $X \cup \{e\}$ is a part of some MST.

Prim's Algorithm

```

X = {}
repeat until |X| = |V| - 1:
    pick a set S in V for which X has no edges between S and V-S
    let e in E be the minimum-weight edge between S and V-S
    X = X and {e}

```

It is also greedy algorithm. And it is similar to Dijkstra algorithm.

```

procedure prim(G,w)
Input: A connected undirected graph G=(V,E) with edge weights
Output: A minimum spanning tree defined by the array prev

for all u in V:
    cost(v) = infinity /*cost(v) = min w(u,v) where u in S*/
    prev(u) = null
Pick any initial node u_0
cost(u_0) = 0

H = makequeue(V) /*priority queue with cost-values as keys*/
while H is not empty:
    v = deletemin(H)
    for each {v,z} in E:
        if cost(z) > w(v,z):
            cost(z) = w(v,z)
            prev(z) = v
            decreasekey(H,z)

```

$O((|V| + |E|) \log |V|)$.

Comparison of Kruskal and Prim

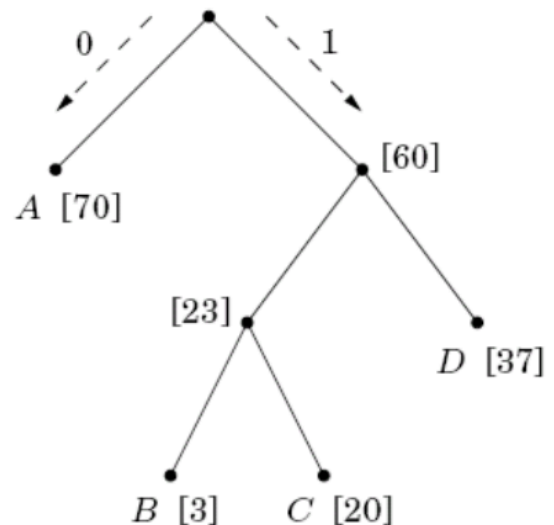
	Kruskal	prim
sort all edges?	yes	no
minimum	the lightest edge in the remaining edges	the lightest edge among the cross edges
data structure	disjoint set (connected component)	binary heap (priority queue)
time complexity	$O((V + E)\log V)$	$O((V + E)\log V)$

Huffman Encoding

Prefix-free encoding:

- Represented by a full binary tree
- Symbols are at the leaves
- Codeword is a path from root to leaf, interpreting left as 0 and right as 1.

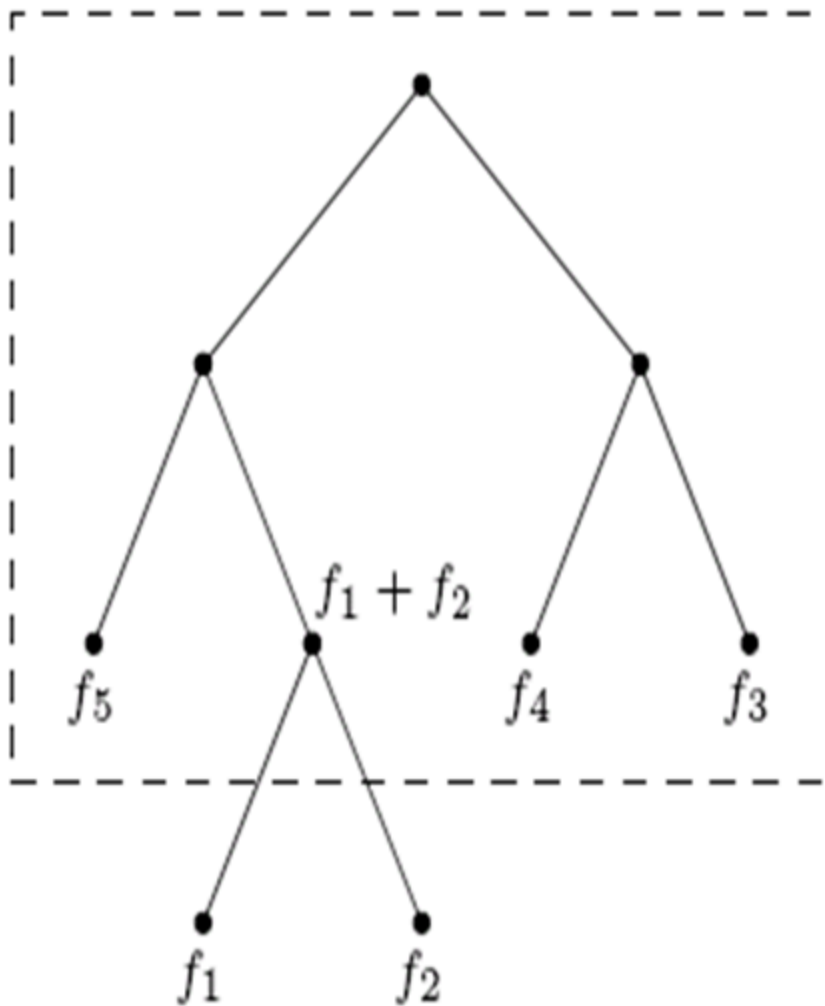
Symbol	Codeword
<i>A</i>	0
<i>B</i>	100
<i>C</i>	101
<i>D</i>	11



To find the optimal coding tree, given the frequencies $f_1, \dots, f_n$ of n symbols, then define the frequency of any internal node to be the sum of the frequencies of its descendant leaves.

Construct the tree greedily:

- Find two symbols with the smallest frequencies f_1, f_2
- Pull f_1 and f_2 off the list of frequencies and insert $f_1 + f_2$ and loop



n leaves and $n - 1$ inner nodes.

```

let H be a priority queue of integres ordered by f[1...n]
for i = 1 to n: insert(H,i) // insert all nodes into heap
for k = n+1 to 2n-1: // initially n nodes for leaves and the last is 2n-1
    i = deleteMin(H), j = deleteMin(H)
    create a node numbered k with children i,j
    f[k] = f[i]+f[j]
    insert(H,k)

```

$O(n \log n)$.

Set Cover Problem

- Input: a set of elements B , sets $S_1, \dots, S_m \subseteq B$
- Output: a selection of the S_i whose union is B

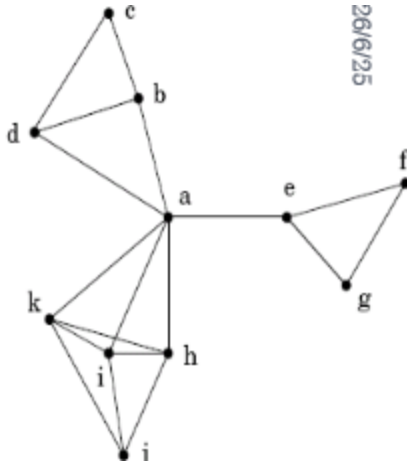
- Cost: number of sets picked

Greedy algorithm:

repeat until all elements of B are covered:

pick the set S_i with the largest number of uncovered elements

For example:



S_a represents $\text{Adj}(a) \cup \{a\}$.

Use greedy algorithm:

- pick S_a , the rest elements are c, j, f, g
 - pick S_c
 - pick S_j
 - pick S_f
- thus a greedy solution is $\{S_a, S_c, S_j, S_f\}$
- An optimal solution is $\{S_b, S_e, S_i\}$.

Suppose B contains n elements and the optimal cover consists of k sets. Then greedy algorithm will use at most $k \ln n$ sets.

To approximate the effect of approximation solution and make it generic for maximum problem, we use approximation factor:

$$\Delta = \max \left\{ \left| \frac{c}{c^*} \right|, \left| \frac{c^*}{c} \right| \right\}$$

where c is the approximation solution and c^* is the optimal solution. The closer the approximation factor is to one, the better the approximation algorithm performs.

Matroid Theory

The theory is used to judge whether the greedy algorithm can obtain optimal solution.

Independent sets, denoted by I , defined in the set S , satisfy the following three conditions

- nonempty. empty set is independent
- hereditary: if A is independent set, then any subsets of A is also independent set.
- exchange property: if A, B are independent sets, and $|A| < |B|$, then there must exist an element $e \in B$ such that $A \cup \{e\}$ is also independent set.

The three conditions can be understood like **linear independence**.

The set system (S, I) is called a matroid.

If a optimization problem can be modeled as a problem that **the largest weighted independent set on the weighted matroid**, the greedy algorithm can find optimal solution.

That is,

- sort the elements in S
- consider each element in order. If the element can be added without destroying the independence of I , add it. If not, drop it.